

Lab 2 Instructor Key: Dynamic Programming in Gridworld

CS/DSA 4XX0 – Reinforcement Learning

Overview

This document is the **instructor key** for Lab 2: Dynamic Programming in Gridworld. It contains:

- Reference pseudocode for all required algorithms.
- The complete Python implementations from `src/dp_solution.py`.
- Notes on expected behavior and grading.

Students only see the student instructions and the TODOs in `src/dp.py`. This key is for instructors and graders.

Environment and Utilities

The Gridworld environment is defined in `src/envs/gridworld.py`. Utilities for printing value functions and policies are in `src/utlis/utlis.py`. Students do not modify these files.

The DP algorithms operate over:

- A state index mapping: `env.state_to_index[(r,c)]` and `env.index_to_state[idx]`.
- A deterministic transition function: `env.get_next_state_and_reward(state, action)`.

Reference Pseudocode

For convenience, here is the pseudocode that matches the instructor implementation.

Policy Evaluation

Policy Improvement

Policy Iteration

Value Iteration

Algorithm 1 Iterative Policy Evaluation

Require: Policy π , discount γ , threshold θ

```
1: Initialize  $V(s) \leftarrow 0$  for all states
2: repeat
3:    $\Delta \leftarrow 0$ 
4:   for each state  $s$  do
5:      $v \leftarrow V(s)$ 
6:      $V(s) \leftarrow \sum_a \pi(a|s) [R(s, a) + \gamma V(s')]$ 
7:      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
8:   end for
9: until  $\Delta < \theta$ 
10: return  $V$ 
```

Algorithm 2 Greedy Policy Improvement

Require: Value function V , discount γ

```
1: for each state  $s$  do
2:   Compute  $Q(s, a) = R(s, a) + \gamma V(s')$  for all actions  $a$ 
3:   Let  $a = \arg \max_a Q(s, a)$ 
4:   Set  $\pi(s)$  to be deterministic:  $\pi(a|s) = 1$ 
5: end for
6: return  $\pi$ 
```

Instructor Implementation: src/dp_solution.py

Below is the complete instructor implementation that matches the pseudocode and the Gridworld environment.

Full Code

```
import numpy as np

def policy_evaluation(env, policy, gamma=0.99, theta=1e-6):
    """
    Iterative policy evaluation.

    env: GridworldEnv with state_to_index and index_to_state
    policy: shape (n_states, n_actions)
    gamma: discount factor
    theta: convergence threshold

    Returns:
    V: shape (n_states,)
    """
    n_states = env.n_rows * env.n_cols
    V = np.zeros(n_states)

    while True:
```

Algorithm 3 Policy Iteration

Require: Discount γ , threshold θ

- 1: Initialize π to be uniform random
 - 2: **repeat**
 - 3: $V \leftarrow \text{PolicyEvaluation}(\pi)$
 - 4: $\pi_{\text{new}} \leftarrow \text{PolicyImprovement}(V)$
 - 5: **until** $\pi_{\text{new}} = \pi$
 - 6: **return** (V, π)
-

Algorithm 4 Value Iteration

Require: Discount γ , threshold θ

- 1: Initialize $V(s) \leftarrow 0$ for all states
 - 2: **repeat**
 - 3: $\Delta \leftarrow 0$
 - 4: **for** each state s **do**
 - 5: $v \leftarrow V(s)$
 - 6: $V(s) \leftarrow \max_a [R(s, a) + \gamma V(s')]$
 - 7: $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 - 8: **end for**
 - 9: **until** $\Delta < \theta$
 - 10: $\pi \leftarrow$ Greedy policy derived from V
 - 11: **return** (V, π)
-

```
delta = 0.0
for s in range(n_states):
    r, c = env.index_to_state[s]

    # Skip terminal states (trap and goal)
    if env.grid[r][c] in ("T", "G"):
        continue

    v = V[s]
    new_v = 0.0

    # Sum over actions according to the policy
    for a, prob in enumerate(policy[s]):
        if prob == 0.0:
            continue
        next_state, reward = env.get_next_state_and_reward((r, c), a)
        new_v += prob * (reward + gamma * V[next_state])

    V[s] = new_v
    delta = max(delta, abs(v - new_v))

if delta < theta:
    break
```

```

    return V

def policy_improvement(env, V, gamma=0.99):
    """
    Greedy policy improvement.

    env: GridworldEnv
    V: value function, shape (n_states,)
    gamma: discount factor

    Returns:
    policy: shape (n_states, n_actions)
    """
    n_states = env.n_rows * env.n_cols
    n_actions = env.action_space.n

    policy = np.zeros((n_states, n_actions))

    for s in range(n_states):
        r, c = env.index_to_state[s]

        # Terminal states: no actions needed
        if env.grid[r][c] in ("T", "G"):
            continue

        q_values = np.zeros(n_actions)

        # Compute Q(s,a) for each action
        for a in range(n_actions):
            next_state, reward = env.get_next_state_and_reward((r, c), a)
            q_values[a] = reward + gamma * V[next_state]

        best_a = np.argmax(q_values)
        policy[s][best_a] = 1.0

    return policy

def policy_iteration(env, gamma=0.99, theta=1e-6):
    """
    Full policy iteration.

    env: GridworldEnv
    gamma: discount factor
    theta: convergence threshold for policy evaluation

    Returns:
    V: optimal value function
    policy: optimal policy
    """

```

```

n_states = env.n_rows * env.n_cols
n_actions = env.action_space.n

# Start with a uniform random policy
policy = np.ones((n_states, n_actions)) / n_actions

while True:
    V = policy_evaluation(env, policy, gamma, theta)
    new_policy = policy_improvement(env, V, gamma)

    # If policy is stable, stop
    if np.array_equal(policy, new_policy):
        break

    policy = new_policy

return V, policy

def value_iteration(env, gamma=0.99, theta=1e-6):
    """
    Value Iteration.

    env: GridworldEnv
    gamma: discount factor
    theta: convergence threshold

    Returns:
    V: optimal value function
    policy: greedy policy derived from V
    """
    n_states = env.n_rows * env.n_cols
    n_actions = env.action_space.n

    V = np.zeros(n_states)

    while True:
        delta = 0.0
        for s in range(n_states):
            r, c = env.index_to_state[s]

            # Terminal states: skip
            if env.grid[r][c] in ("T", "G"):
                continue

            v = V[s]
            q_values = np.zeros(n_actions)

            # Compute Q(s,a) for each action
            for a in range(n_actions):
                next_state, reward = env.get_next_state_and_reward((r, c), a)

```

```

        q_values[a] = reward + gamma * V[next_state]

    V[s] = np.max(q_values)
    delta = max(delta, abs(v - V[s]))

    if delta < theta:
        break

policy = policy_improvement(env, V, gamma)
return V, policy

```

Expected Behavior

When running `python -m src.main_solution` from the repository root, the instructor version should:

- Print the value function and policy from Policy Iteration.
- Print the value function and policy from Value Iteration.
- Report that the maximum difference between the two value functions is effectively zero.

The autograder `tests/test_lab2.py` can be temporarily pointed at `dp_solution` to verify correctness:

```

from src.dp_solution import (
    policy_evaluation,
    policy_improvement,
    policy_iteration,
    value_iteration,
)

```

All four tests should pass.

Grading Notes

- Students are graded primarily on `src/dp.py`.
- Common mistakes include:
 - Forgetting to skip terminal states in updates.
 - Using the wrong state index mapping.
 - Not updating Δ correctly in iterative algorithms.
 - Failing to construct a deterministic greedy policy in `policy_improvement`.
- Partial credit can be awarded for:
 - Correct structure but incorrect convergence.
 - Correct policy evaluation but incorrect improvement.
 - Correct value iteration but missing policy extraction.